

TorchTitan-NPU:

训练性能与易用性齐飞，开启昇腾训练入图新体验

作者：曹靖宜

时间：2026.05

目录

Part 1 TorchTitan介绍

Part 2 核心特性介绍

Part 3 DFX & 易用性

Part 4 训练入图性能

Part 5 QuickStart & Roadmap

为什么选择 TorchTitan

原生分布式编程模型

- 声明式的统一抽象语义，**DTensor**支持Shrad / Replicate / Partial 等Placement语义。
- **DTensor**基于完整 DeviceMesh，可以低侵入组合 **FSDP + TP + PP + EP + CP**，实现5D并行，保持代码紧凑、模块化。
- 模型切分与模型定义解耦，开发者主要声明子模块 ParallelStyle 来定义如何切分，**无需手写通信逻辑**。

PyTorch Native训练，畅享"全家桶"

- TorchTitan 是 PyTorch 社区原生大模型训练框架，便于**复用开源方案**并降低迁移门槛。
- 原生支持 **torch.compile + Inductor**，通过 FX 图变换、模式匹配和算子融合减少冗余开销。
- 将 PyTorch 多项原生能力(**DTensor、Compile、DCP、TorchAO、TorchFT**) 以系统化的方式有机整合，而非碎片化调用

TorchTitan-NPU插件化支持方案



免侵入式插件机制



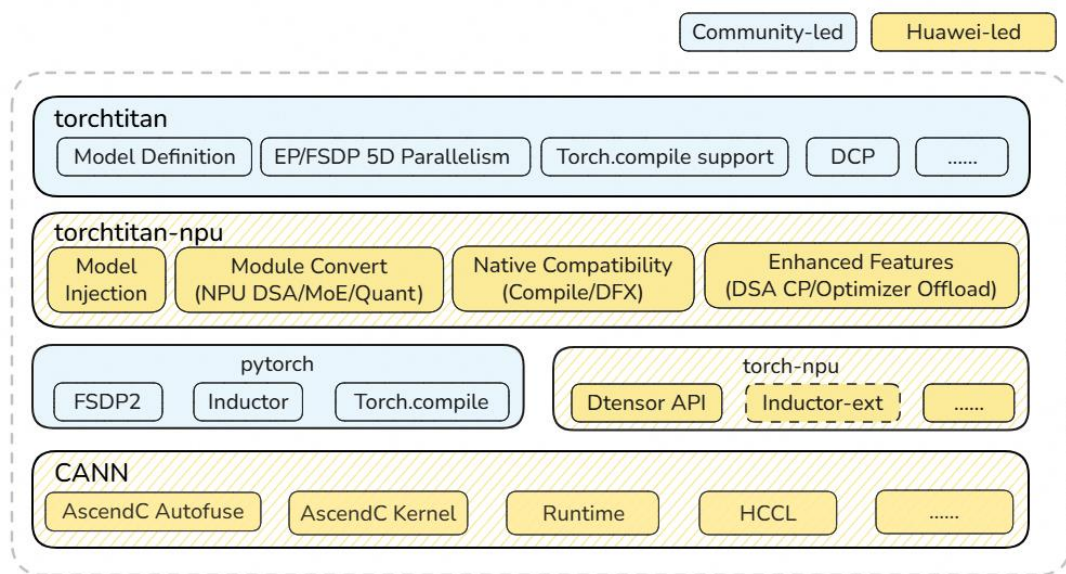
按需启用的开关配置



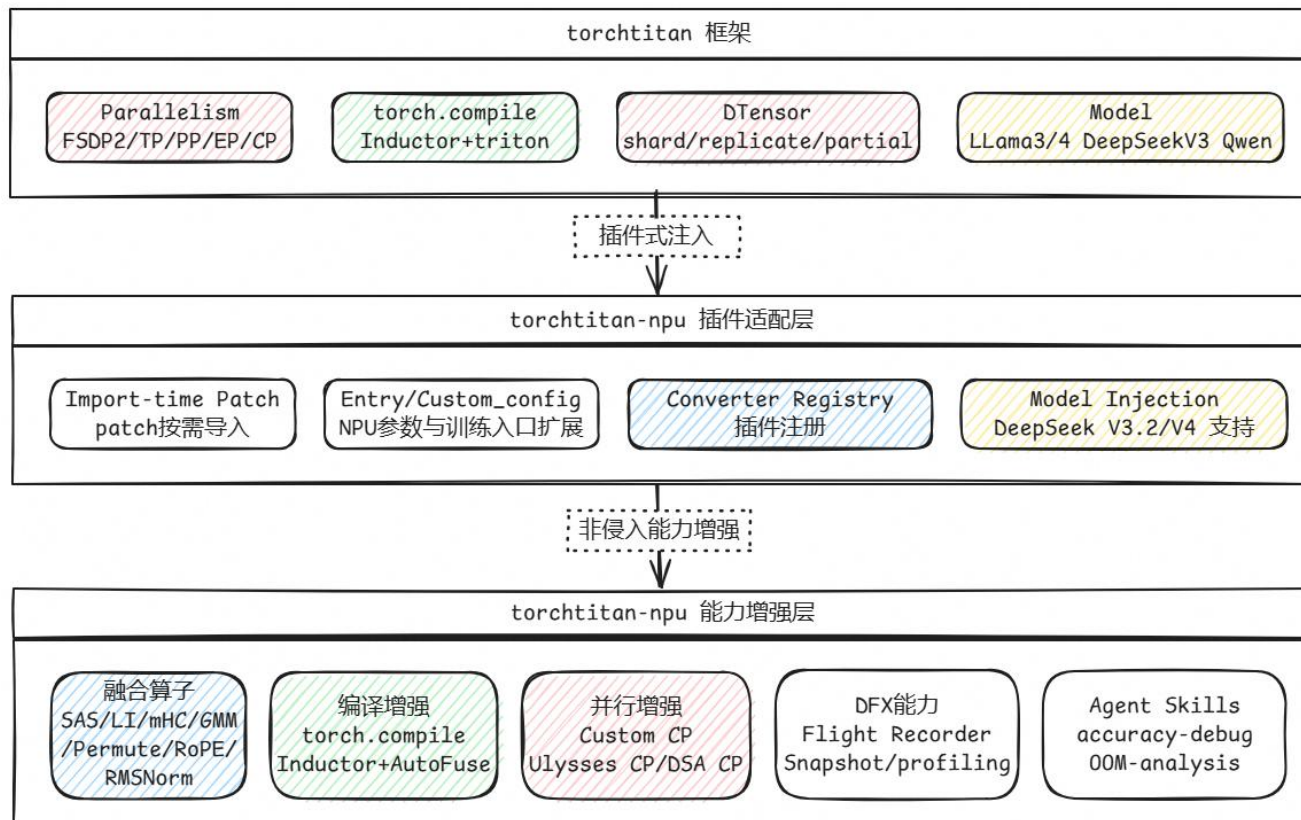
自由组合的优化矩阵



无缝对接 torch.compile+inductor



<https://gitcode.com/cann>



CANN

目录

Part 1 TorchTitan介绍

Part 2 核心特性介绍

Part 3 DFX & 易用性

Part 4 训练入图性能

Part 5 QuickStart &Roadmap

基础特性 - 融合算子支持

➤ 训练文件 converters 配置生效

```
[model]
name = "deepseek_v32"
flavor = "debugmodel"
hf_assets_path = "./assets/hf/DeepSeek-V3.2"
converters = ["npu_dsa", "npu_rms_norm", "npu_permute", "npu_gmm", "npu_expert_parallel"]
```

➤ 已支持的融合算子

- DSA (DeepSeek Sparse Attention)
- GMM (Grouped MatMul)
- Permute
- RMSNorm
- RoPE

converters配置	使能NPU融合算子
npu_dsa	npu_lightning_indexer npu_sparse_lightning_indexer_grad_kl_loss npu_sparse_flash_attention
npu_rms_norm	npu_rms_norm
npu_permute	npu_moe_token_permute npu_moe_token_unpermute
npu_gmm	npu_grouped_matmul
npu_rope	npu_rotary_mul



根据用户配置，使能 NPU 融合算子实现，实现模型在 NPU 上的亲和支持。

基础特性 - 融合算子收益

➤ DSA算子

裁剪模型	实现方式	Memory(G)	Throughput (tokens/p/s)
8层裁剪模型	原始小算子实现	58.32	155
8层裁剪模型	矩阵吸收 + DSA融合算子	50.54	400
4层裁剪模型	原始小算子实现	54.58	402
4层裁剪模型	矩阵吸收 + DSA融合算子	38.71	568

- 8层裁剪模型(1个Dense层, 7个MoE层, 每个MoE层8个专家)
- 4层裁剪模型(1个Dense层, 3个MoE层, 每个MoE层8个专家)
- DeepSeek-V3.2 模型在使能DSA融合算子后, 可以显著降低内存占用, 同时提升计算速度。

➤ 常用融合算子(device执行耗时)

算子类型	融合前耗时	融合后耗时	时间收益	性能提升比例
RMSNorm	138.7μs	39.9μs	98.8μs	71.2%
Permute	803.9μs	127.3μs	676.6μs	84.2%
GMM	3.1ms	2.4ms	0.7ms	22.6%

- 通过将多个细粒度的小算子合并为单一算子, 显著减少算子启动开销, 提升NPU利用率
- 开启融合算子后, DeepSeek-V3 模型训练端到端耗时减少**24%**

基础特性 – 低精度训练支持

- 传统的 BF16/FP16 混合精度训练虽然已大幅降低了显存占用，但在超大规模模型上仍面临计算效率瓶颈。
- TorchTitan-NPU 中引入了对 MXFP8 和 HiFloat8 两种 8-bit 浮点格式的支持，覆盖普通线性层(Linear) 和 MoE 专家层两大场景
- 支持 Ascend **950** 及更高架构的 NPU 设备

模块	配置	功能描述	核心算子
Linear	recipe_name: MXFP8	对激活和权重进行 per-block 量化 (block size=32)，执行低精度矩阵乘法。	torch_npu.npu_dynamic_mx_quant torch_npu.npu_quant_matmul
	recipe_name: HiFloat8	对激活和权重进行 per-tensor 动态量化，整个张量共享一个 scale	torch_npu.npu_dynamic_quant torch_npu.npu_quant_matmul
	filter_fqns	指定不进行低精度替换的线性层	
MoE	recipe_name: MXFP8	对激活和权重进行 per-block 量化 (block size=32)，执行低精度矩阵乘法。	torch_npu.npu_dynamic_mx_quant torch_npu.npu_grouped_matmul
	recipe_name: HiFloat8	先根据专家数和 EP 并行度计算分组大小，对每个专家分组独立量化，每个专家分组共享一个scale	torch_npu.npu_dynamic_quant torch_npu.npu_grouped_matmul

示例一：仅对线性层启用低精度训练

```
[job]
custom_config_module = "torch titan_npu.config.custom_config" # 使能本代码仓的自定义配置

[model]
converters = ["quantize.linear.mx"]

[quantize.linear.mx]
recipe_name = "mxfp8" # 可选 "mxfp8" 或 "hif8"
filter_fqns = ["output", "router.gate"] # output 和 router.gate 层不做低精度替换
```

示例二：同时对线性层和 MoE 专家层启用低精度训练

```
[job]
custom_config_module = "torch titan_npu.config.custom_config"

[model]
# npu_gmm 必须在 quantize.grouped_mm.mx 之前
converters = ["npu_gmm", "quantize.linear.mx", "quantize.grouped_mm.mx"]

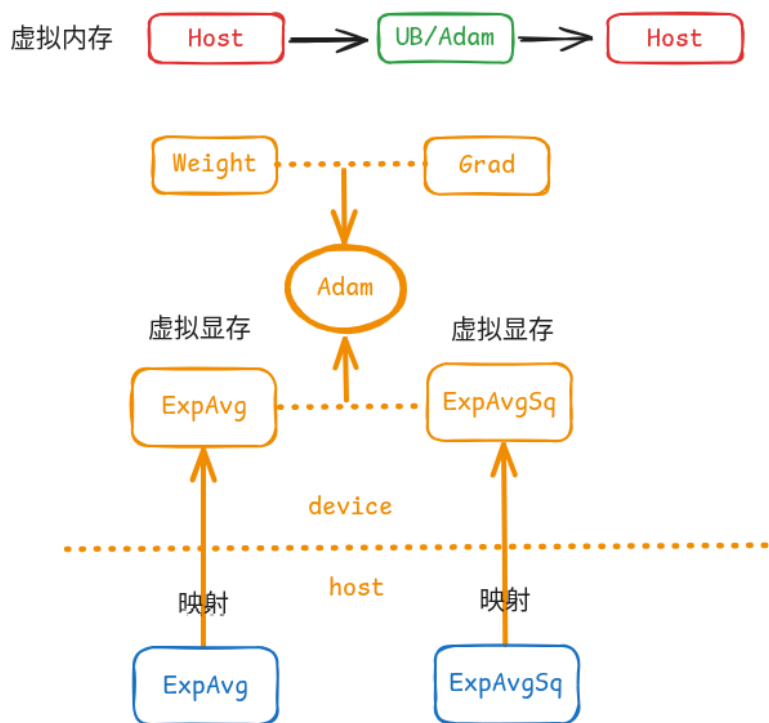
[quantize.linear.mx]
recipe_name = "mxfp8" # 可选 "mxfp8" 或 "hif8"
filter_fqns = ["output", "router.gate"]

[quantize.grouped_mm.mx]
recipe_name = "mxfp8" # 可选 "mxfp8" 或 "hif8"
fqns = ["experts"]
```



增强特性 – Virtual Optimizer

Virtual Optimizer(虚拟优化器)



配置示例

首先在配置文件中使能本代码仓的自定义配置，随后在 [optimizer] 节中添加以下配置，为AdamW优化器开启SwapOptimizer特性并设置流水线切片参数：

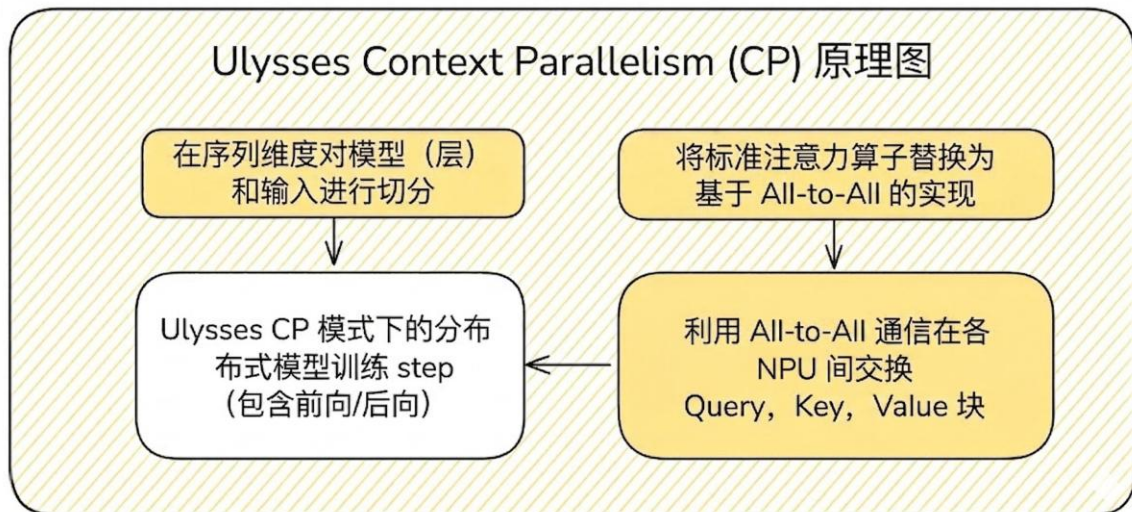
```
[job]
custom_config_module = "torchtitan_npu.config.custom_config" # 使能本代码仓的自定义配置

[optimizer]
name = "AdamW"
lr = 3e-4
weight_decay = 0.01
virtual_optimizer = true # 启用 Virtual Optimizer
virtual_optimizer_size = 'all' # 设置申请的虚拟内存大小
```

- Virtual Optimizer 是一套基于 Ascend NPU 虚拟内存能力的优化器显存优化方案。
- 核心思想：**充分利用 Ascend 驱动原生的虚拟内存映射能力，申请一个实际内存在 Host 侧但地址可被映射到 Device 上的张量，使其可参与大多数 NPU 算子计算。

增强特性 - 自定义CP

SDPA Ulysses CP —— Llama / DeepSeekV3

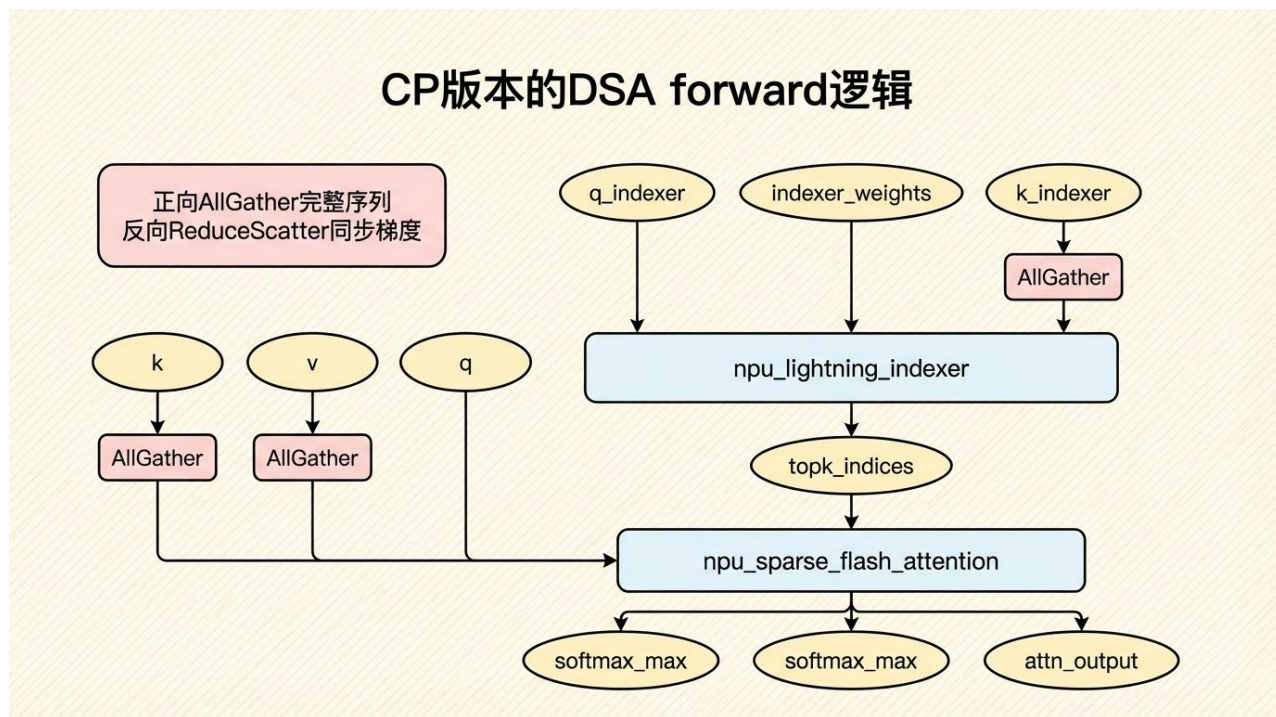


主要问题:

- PyTorch 原生 CP 设计强绑定于标准的 SDPA 算子
- 框架需要允许开发者灵活扩展新的 CP 范式

<https://gitcode.com/cann>

DSA的自定义CP —— DeepSeek-V3.2



TorchTitan-NPU

- 实现了 CustomContextParallelContext 基类
- 用户可在子类中自定义模型前向计算的CP patch逻辑

CANN

目录

Part 1 TorchTitan介绍

Part 2 核心特性介绍

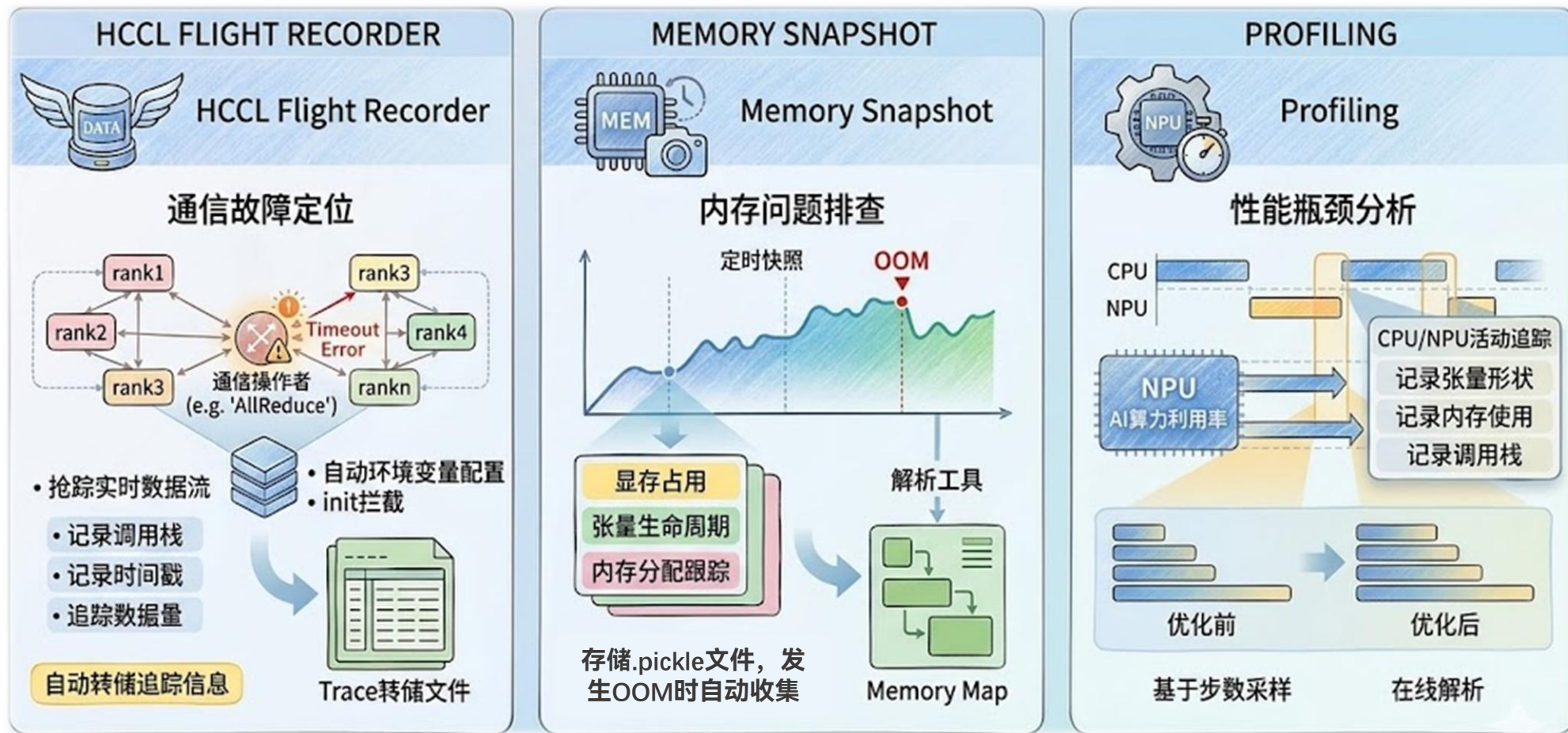
Part 3 DFX & 易用性

Part 4 训练入图性能

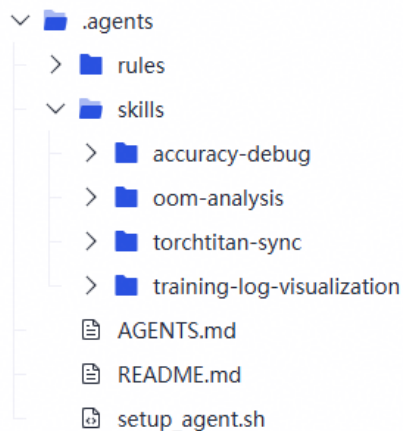
Part 5 QuickStart &Roadmap

DFX & 易用性 — DFX支持

➤ TorchTitan-NPU 目前提供多种调试特性支持，包括通信故障、内存问题和性能瓶颈等。



DFX & 易用性 — Agent能力



快速开始

1. 在仓库根目录执行:

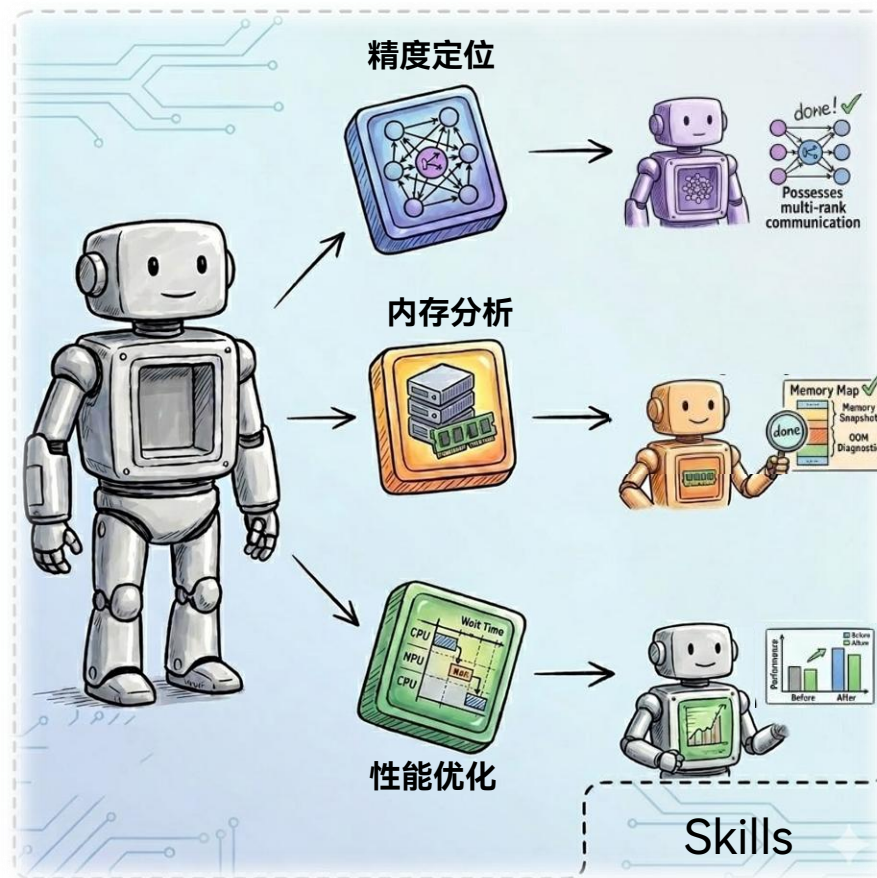
```
bash .agents/setup_agent.sh
```

2. 在 `torchtitan-npu` 开发环境中发起会话。

3. 调用 skill (例如 `/skills` 选择对应 skill, 或通过关键词自动触发)。

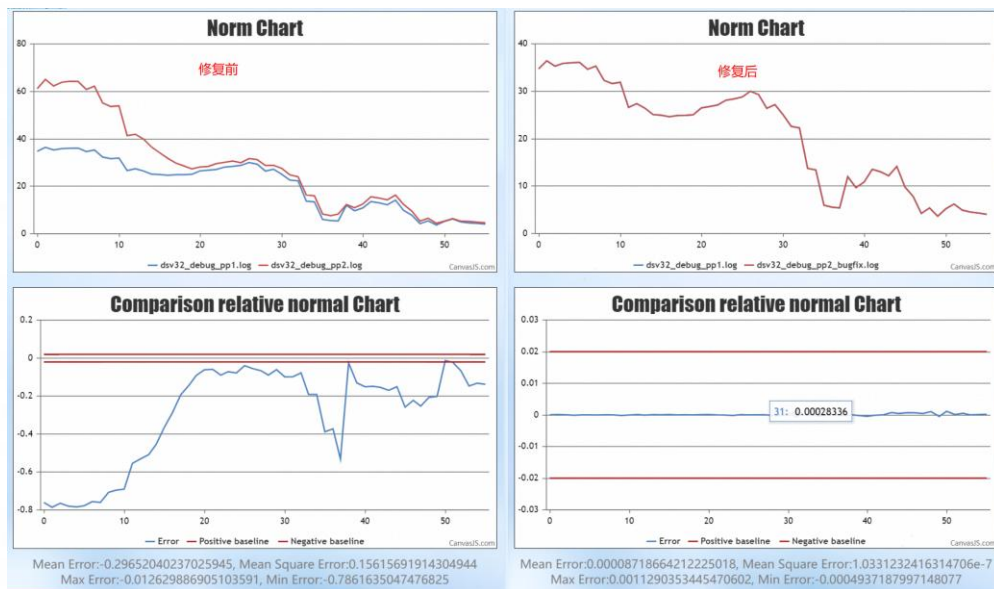
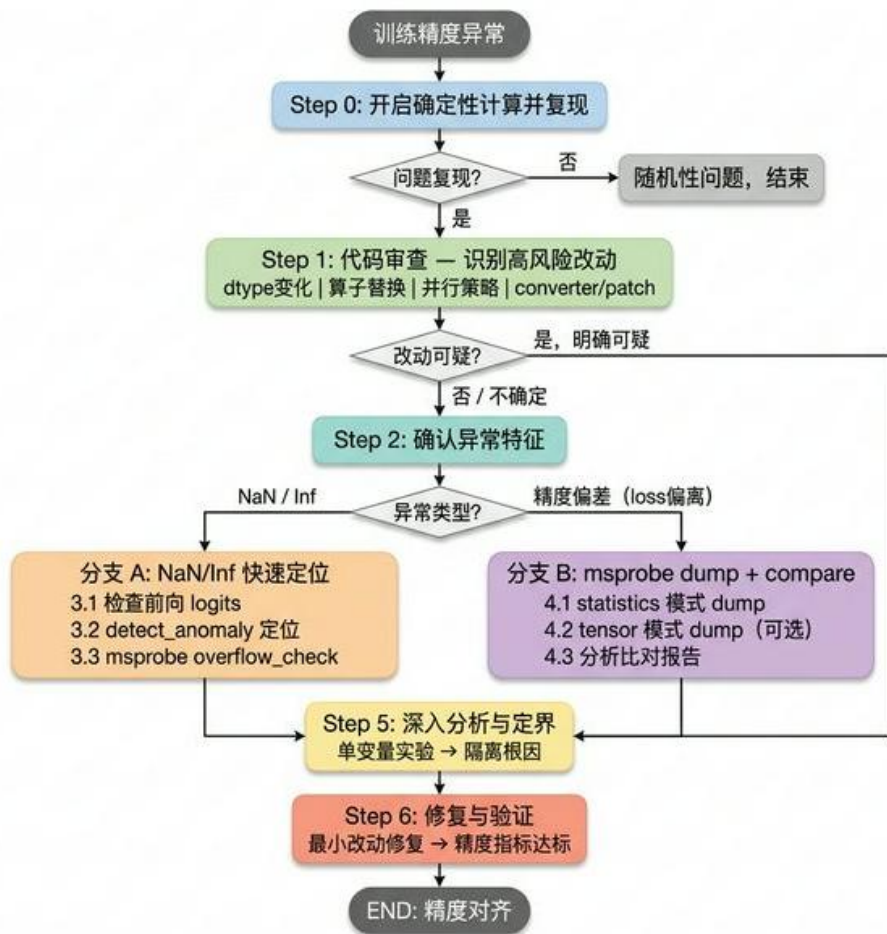
基于昇腾NPU的训练调试经验总结, 支持Agent快速定位和解决问题

- 训练精度异常定位
- OOM问题诊断
- 上游TorchTitan分支同步与适配
- 训练日志可视化



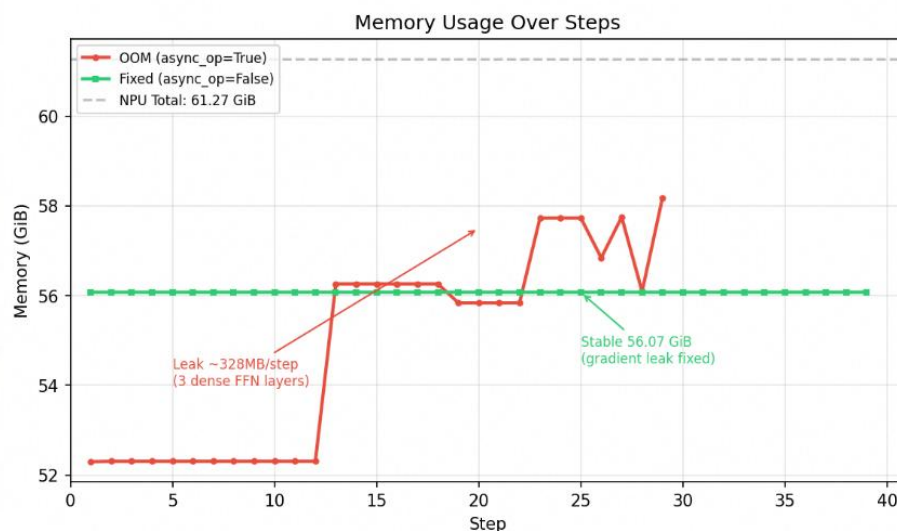
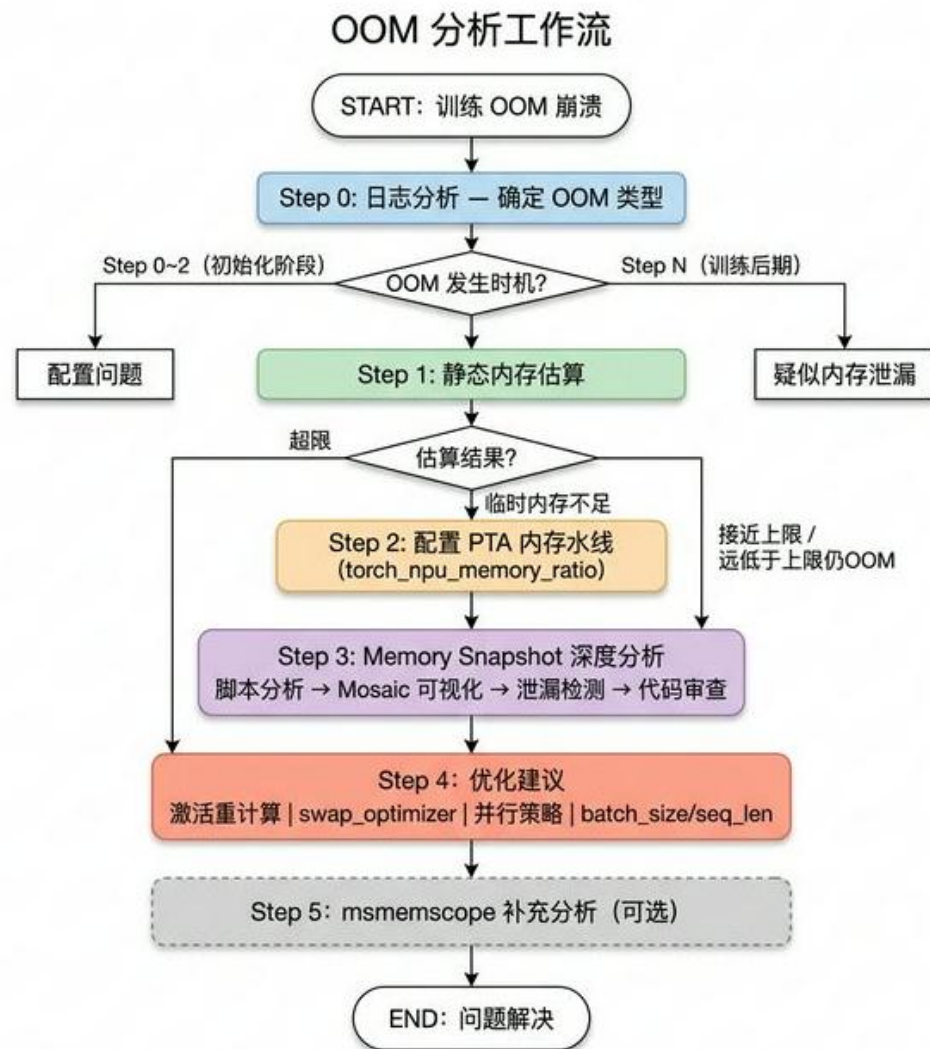
DFX & 易用性 — Agent能力

精度异常分析 workflow



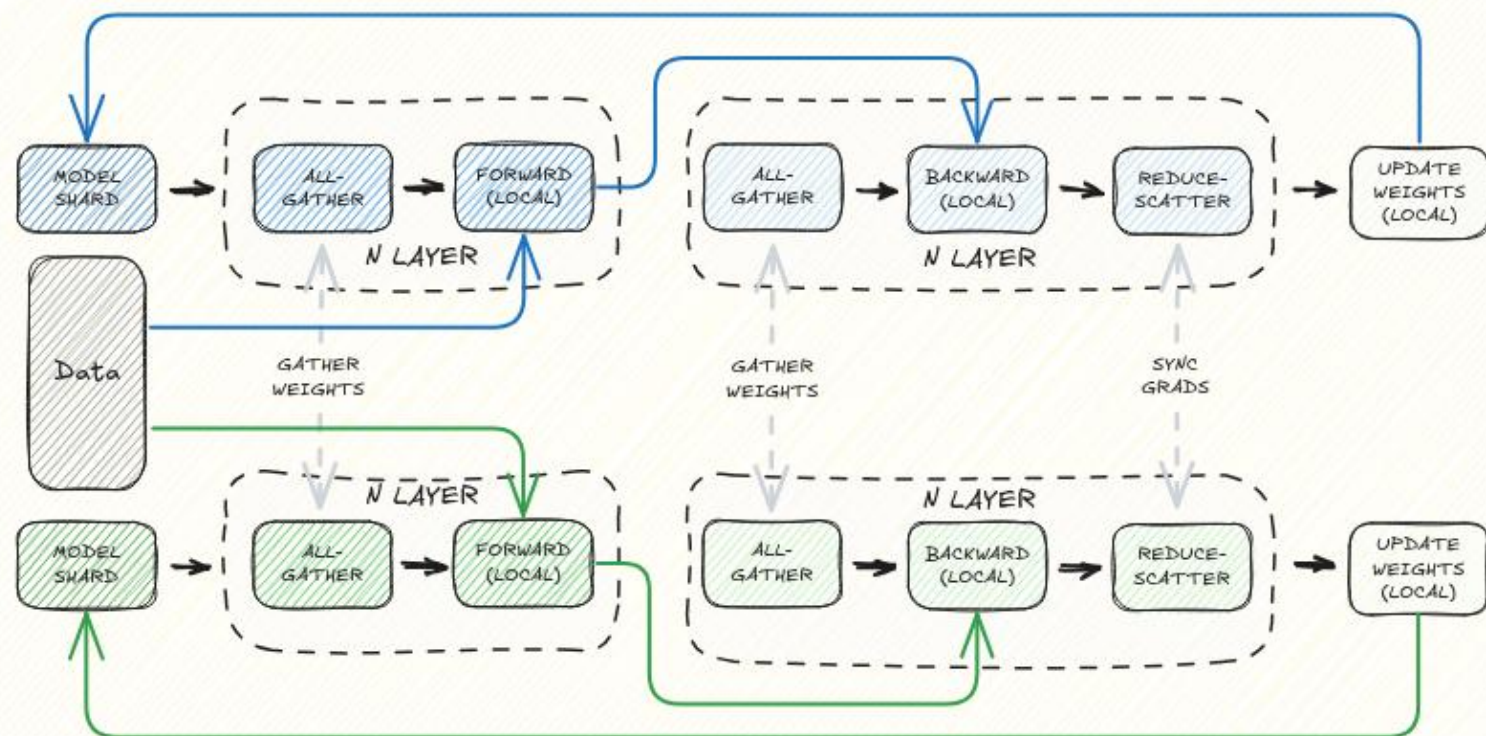
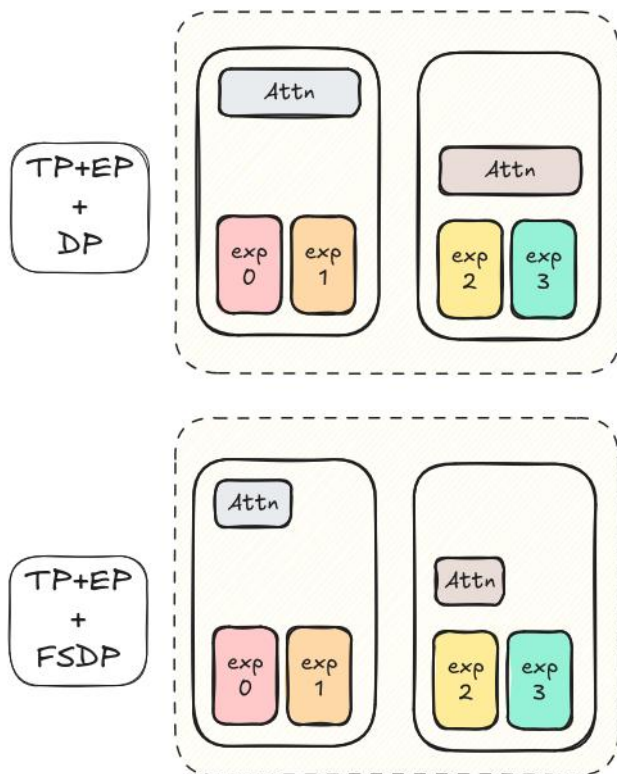
- 案例：在开启pp且lbs>1的场景下，indexer loss grad norm与不开启pp场景无法对应
- agent给出定位结论并且给出问题修复的最小补丁
- Claude Code + GPT 5.3 定位仅**7分钟**，人工定位0.5天

DFX & 易用性 — Agent能力

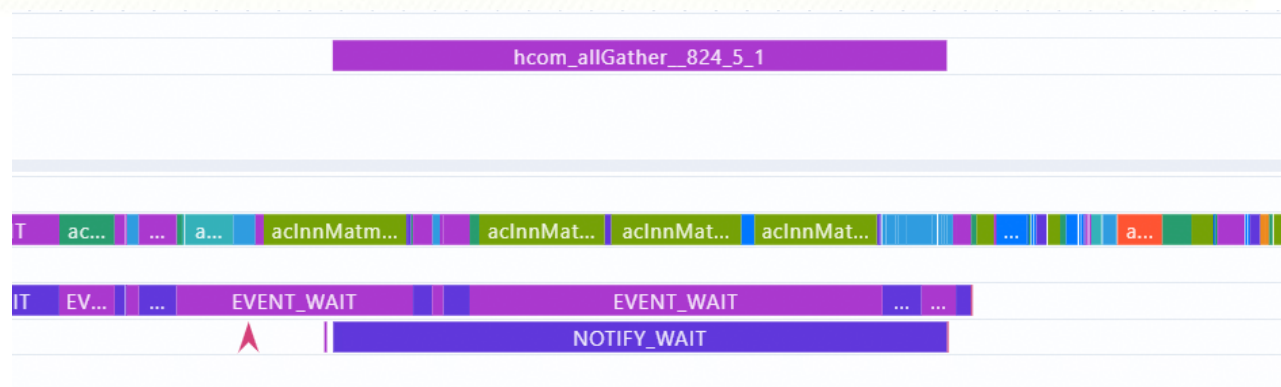


- OOM类型判断 → snapshot 采集 → 语义化分析 → 配置级修复建议
- 案例：针对 DeepSeek-V3.2 模型开启完全重计算与 TP、且含 dense 层时的内存泄漏 OOM 场景
- Agent 结合报错日志和采集memory snapshot快速完成根因定位
- Agent 定位仅**20分钟**，人工定位3天

DFX & 易用性 — FSDP2简化并行



- PP流水线bubble调优困难 & stage不均衡问题
- 使用(TP)+EP+FSDP2更简单的并行方案 => 更易用
- 零冗余权重部署 => 更小的内存占用
- A3超节点高通信带宽 => 更多的通信掩盖



目录

Part 1 TorchTitan介绍

Part 2 核心特性介绍

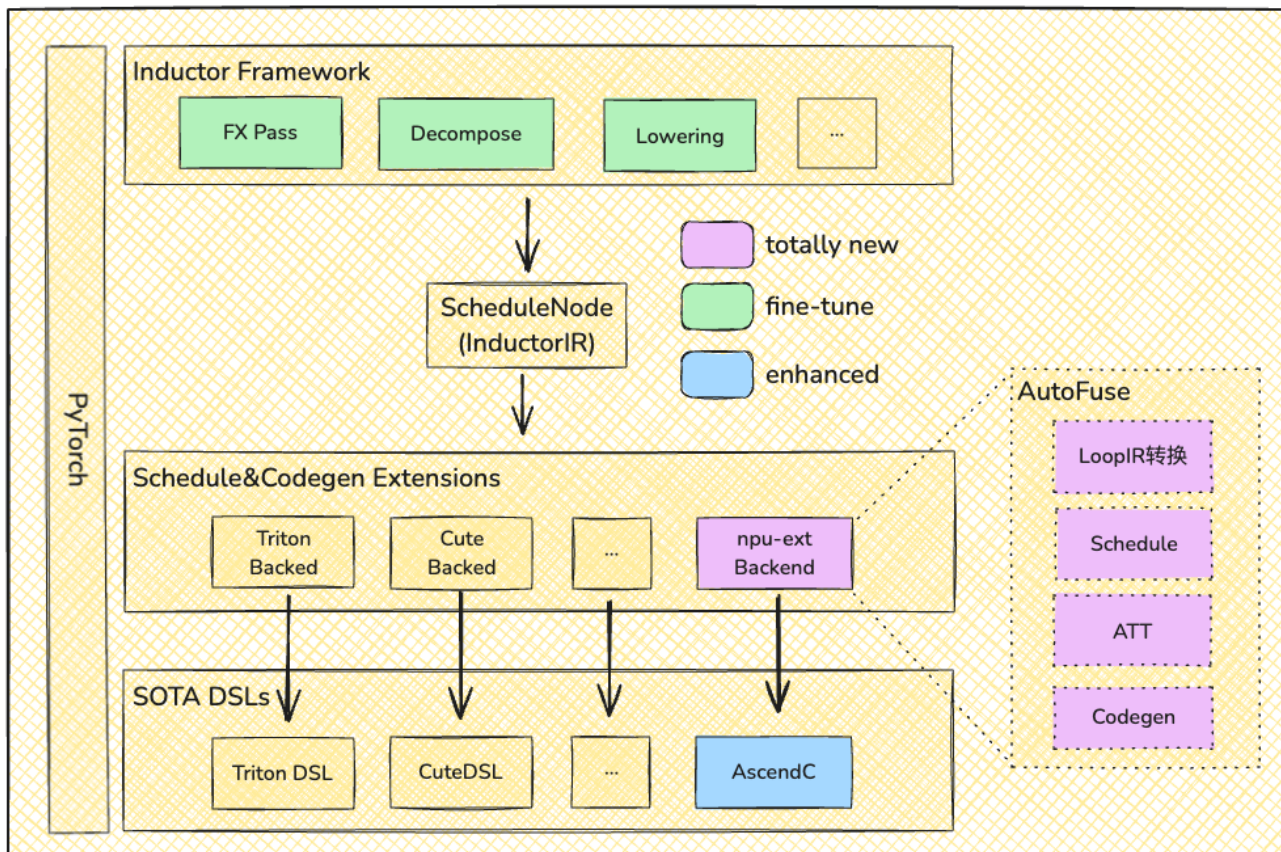
Part 3 DFX & 易用性

Part 4 训练入图性能

Part 5 QuickStart & Roadmap

Torch.compile + Inductor接入方案

从 FX 图捕获到 AutoFuse 自动生成 Ascend C Kernel



一行使能 & 极简易用

PyTorch 前端增加一行代码即可开启入图优化。

NPU 亲和的 Codegen

自动生成硬件架构感知的 Ascend C Kernel

性能收益

Host-Bound 消减; 访存开销消减; 前端图变换消除无效算子

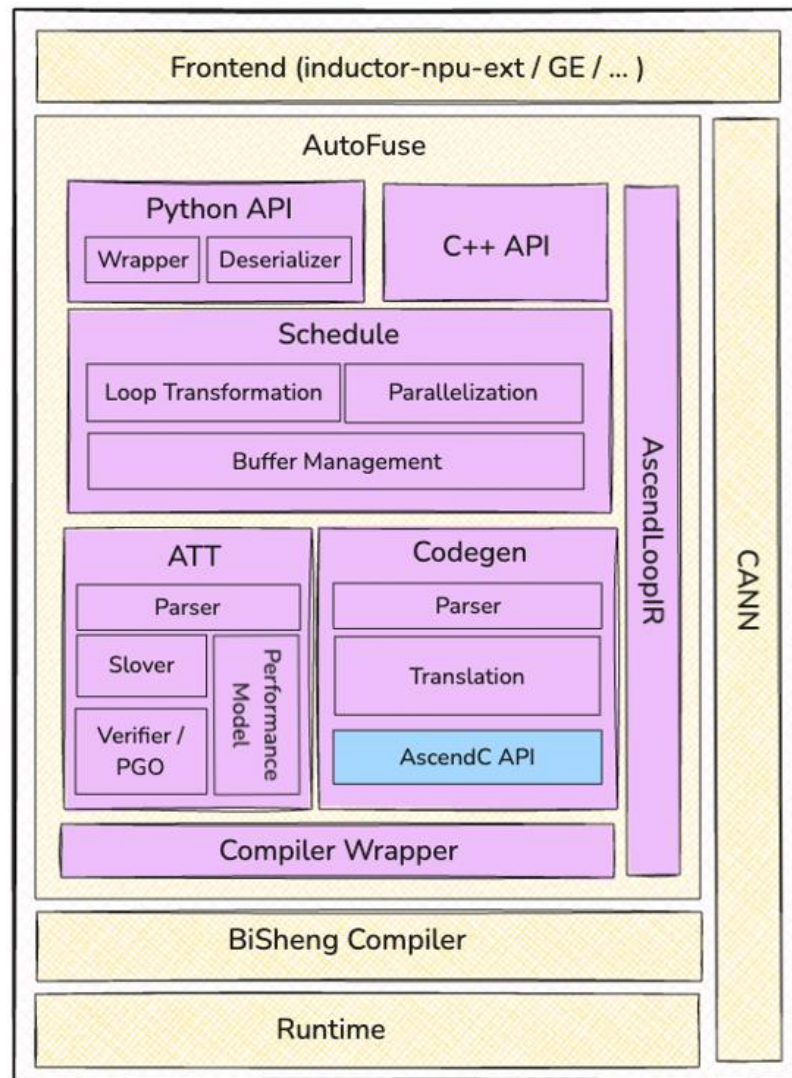
Inductor + AutoFuse: 使能方式

- TorchTitan-NPU 在保留Dynamo和Inductor既有编译流程基础上，接入CodeGen后端 **inductor_npu_ext**
- 基于AutoFuse自动融合能力，从 Inductor IR 生成 Ascend C 融合算子

➤ 训练文件 compile 配置生效

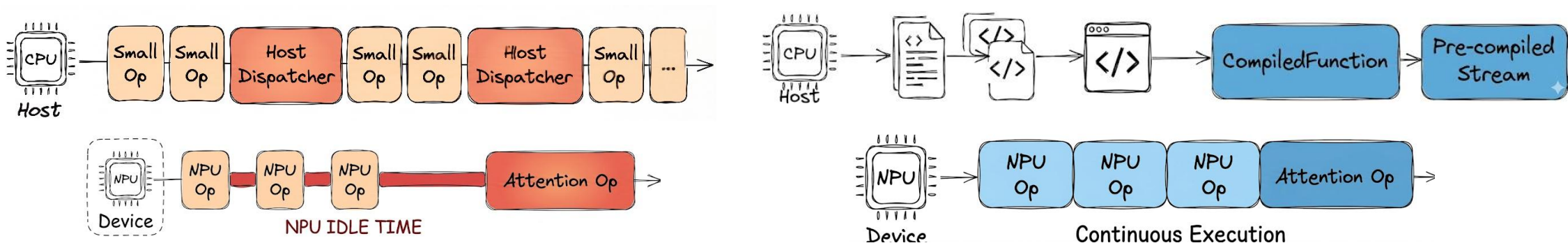
```
[compile]
# 启用编译
enable = true
# 编译完整模型，而不是只编译 loss 。
components = ["model", "loss"]
```

```
try:
    # pyrofly: ignore [missing import]
    import inductor_npu_ext # noqa: F401
except Exception as e:
    raise RuntimeError(
        f"compile.enable is True for {config.model.name} model but inductor_npu_ext is not available. "
        "Please install inductor_npu_ext before enabling compile. "
        "See docs/torch_compile.md for installation instructions."
    ) from e
```



Inductor + AutoFuse: 性能优化

🔊 Host-Bound 调度瓶颈消减



📊 Eager模式 Dispatch和调度瓶颈

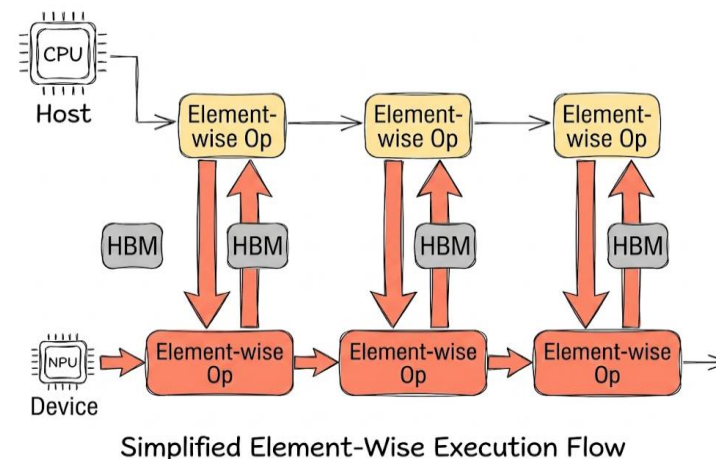
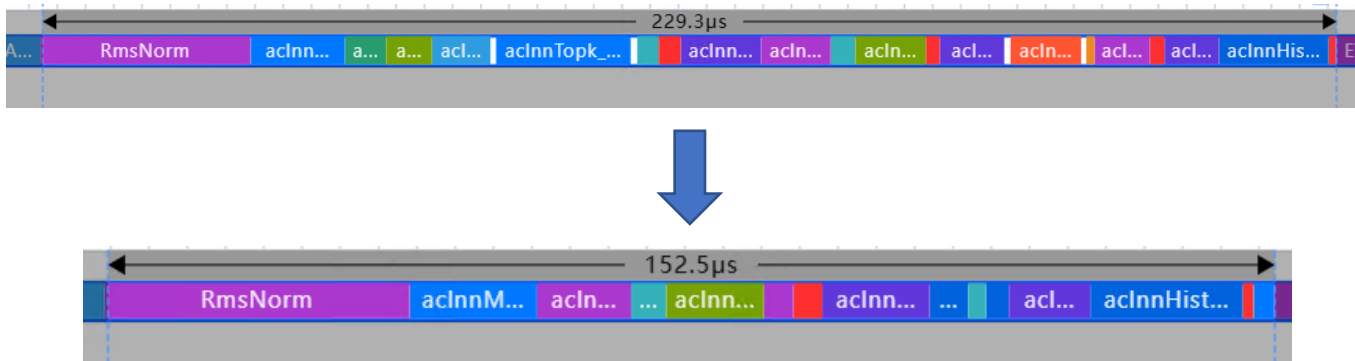
🚀 Inductor模式 降低调度开销

- Inductor 通过静态化分析与Codegen Wrapper，将动态调度转化为预编译的执行流
- 绕过了昂贵的 PyTorch Dispatcher 机制，单层仅在 Attention 计算前就消除了~2ms的 Host 延迟
- NPU 计算流水线得以无缝衔接，大幅提升了硬件利用率

Inductor + AutoFuse: 性能优化

Memory-Bound 访存开销消减

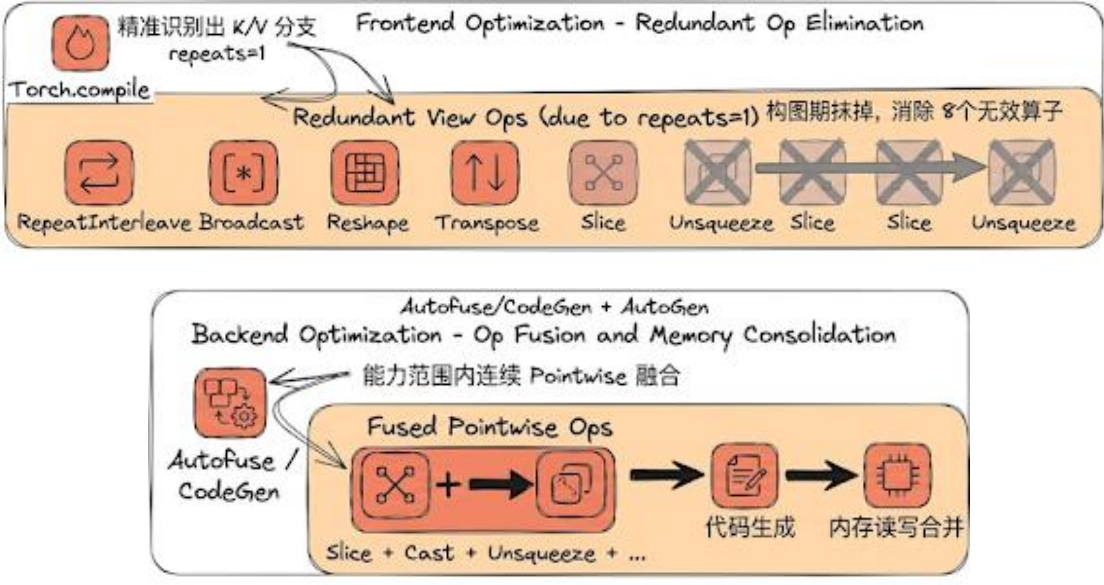
- DeepSeek-V3 的 MoE 路由层与归一化层包含大量受限于显存带宽的小算子



- 每一个独立的 Element-wise 操作，在 Eager 模式下必须对 HBM(显存) 进行完整的读写，极易触发带宽瓶颈。
- AutoFuse 利用 **垂直融合** (Vertical Fusion) 将多个相邻的小算子融合成一个 Kernel，中间变量仅在寄存器中流转。
- 对于Top K计算部分计算耗时优化229.3us -> **152.5us**

Inductor + AutoFuse: 性能优化

前端图变换消除冗余逻辑

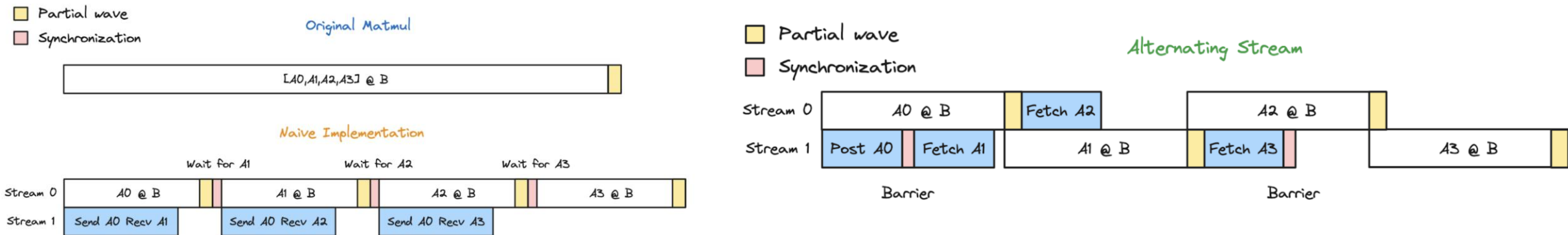


Eager	Inductor + AutoFuse
MatMulV2	MatMulV2
Slice	
Cast	
StridedSlice	
BroadcastTo	
RepeatInterleaveV2	
StridedSlice	
BroadcastTo	
RepeatInterleaveV2	
RotaryPositionEmbedding	RotaryPositionEmbedding
Slice	Slice
RmsNorm	RmsNorm
MatMulV2	MatMulV2

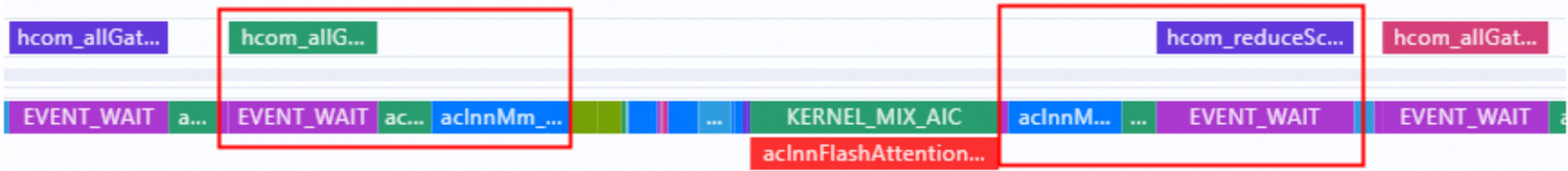
- 前端图变换: 精准识别出 K/V 分支 中因为 repeats=1 导致的冗余视图操作
- 后端算子融合: 对连续 Pointwise 算子 (Slice + Cast + Unsqueeze 等) 进行了代码生成和内存读写合并
- Rope部分计算时间从126us->15us

Inductor + TPAsync

- Async TP 的核心思想是将通信与计算算子同时进行分解，使得一个子 Matmul 的计算与下一个 chunk 的数据传输并发执行，从而隐藏通信延迟
- TorchTriton-NPU使用通算融合算子 (Compute-Communication Fusion)将原本分离的通信和计算内核合并为一个单一的融合算子内核



➤ 开启前



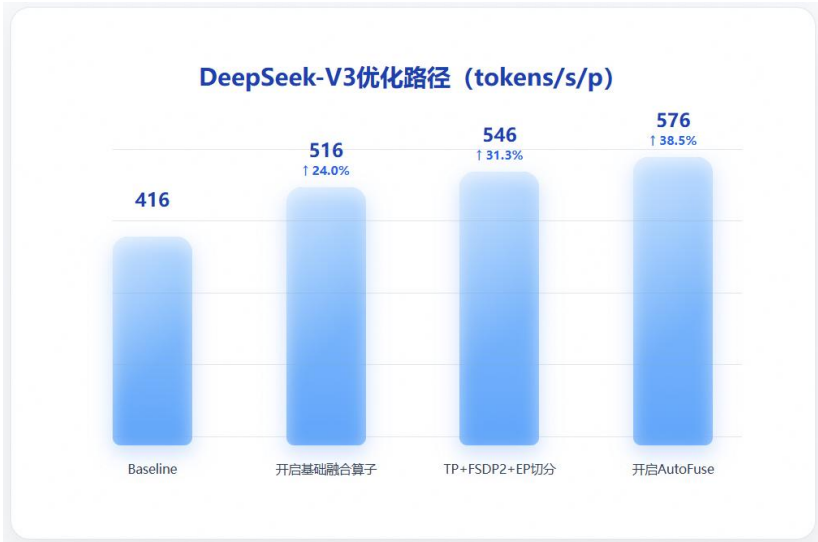
➤ 开启后



训练性能评估

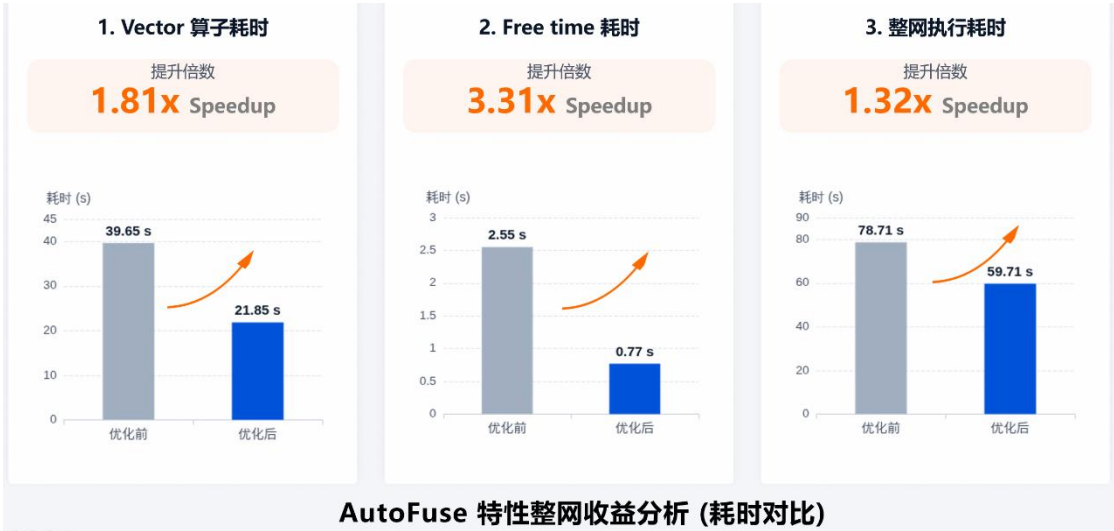
- 依托CANN平台+ TorchTitan-NPU插件，在A3 64卡集群上完成了DeepSeek-V3/V3.2/V4完整模型继续预训练
- DeepSeek-V3模型通过使能融合算子，使用TP+EP+FSDP2切分，使能AutoFuse，相比开箱性能提升**38.5%**，整网吞吐达到**576**tokens/p/s

Model	Number of NPUs	Precision	GBS	Local BS	Sequence Length	FSDP	TP	PP	CP	EP	Throughput (tokens/p/s)
DeepSeek-V4-Flash	64	BF16	1024	1	4096	128	1	1	1	128	1100
DeepSeek-V3.2-671B	64	BF16	128	1	32768	4	4	1	8	64	103
DeepSeek-V3.2-671B	64	BF16	512	1	4096	32	4	1	1	64	146
DeepSeek-V3-671B	64	BF16	1024	1	4096	32	4	1	1	128	546
DeepSeek-V3-671B + compile(Autofuse)	64	BF16	1024	1	4096	32	4	1	1	128	576



- DeepSeek-V4 mHC结构 AutoFuse有更好表现

block	mhc小算子	mhc triton		mhc AF融合	
	耗时(us)	耗时(us)	加速比	耗时(us)	加速比
hc_pre	3814.4	2262.9	1.69	2474.4	1.54
attn	10291.5	9800.8	1.05	9587.6	1.07
hc_post	3303.3	3055.9	1.08	1100.3	3.00
hc_pre	3078.1	1083.5	2.84	1945.6	1.58
moe	26374.2	25009.6	1.05	23095.1	1.14
hc_post	3897.6	2635.8	1.48	1061.3	3.67
total	50759.1	43848.6	1.16	39264.3	1.29



目录

Part 1 TorchTitan介绍

Part 2 核心特性介绍

Part 3 DFX & 易用性

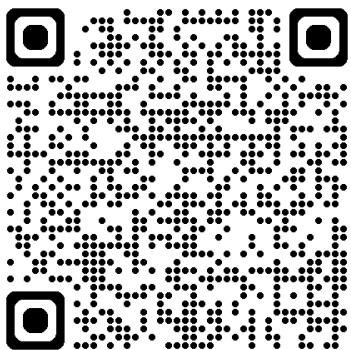
Part 4 训练入图性能

Part 5 QuickStart & Roadmap

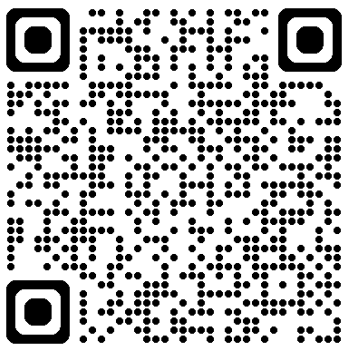
一站式平台体验

- 跟随ReadMe说明，使用一站式平台体验Qwen3-0.6B模型CPT训练
- 只需完成简单环境配置即可一键开启TorchTitan训练体验

- **性能打榜挑战**
 - 内存优化
 - 性能调优



一站式平台体验



打榜活动入口



打榜目标：在 loss 收敛与基线一致（前 100 steps loss 曲线相对偏差 $< 1\%$ ）的前提下，刷新以下两个赛道的基线纪录。同时改进双指标的提交可同时进入两个榜单。

显存榜：device内存峰值↓

- **排序指标：**device内存峰值占用（GB），越低越靠前
- **非退化约束：**稳态单 step 时延相对基线退化 $\leq 5\%$

时延榜：单 step 时延↓

- **排序指标：**稳态单 step 时延（s），越低越靠前
- **非退化约束：**device内存峰值相对基线退化 $\leq 5\%$

可探索的优化方向

- **多维并行：**开启 Tensor Parallel（`tensor_parallel_degree > 1`）或 Pipeline Parallel（`pipeline_parallel_degree > 1`），探索通信 / 计算 overlap；
- **Context Parallel：**长序列场景下叠加 CP（`context_parallel_degree > 1`），需对 RoPE 实现做配套适配；
- **图编译：**使能 `torch.compile` + AutoFuse，验证 dynamo 在 Qwen3 路径上的兼容性与端到端收益；
- **异步 Checkpoint：**将 `async_mode` 切换至 `async_with_pinned_mem`，降低落盘对训练 step 的阻塞；
- **Activation Checkpoint 策略：**在 `selective_ac_option` 上探索更细粒度的算子选择，权衡显存与重算开销。

Future Plan

- TorchTitan-NPU 目前已在TorchTitan支持模型基础上，新增 DeepSeek-V3.2/DeepSeek-V4-Flash模型预训练支持
- 特性&能力规划：
 - 完善模型支持 (DeepSeek-V4-Pro)
 - 训练特性支持 (Muon优化器)
 - 整网开箱性能优化
 - SFT能力支持
 - 支持RL训练使用torch titan后端
- TorchTitan-NPU 持续丰富和完善 CANN 在大模型训练场景下的 开源生态版图。

26年Q2概览

本季度 torchtitan-npu 聚焦增强基础编译能力与性能优化；同时探索 torchtitan 作为RL训练后端的可能。

特性与能力

状态	特性	支持状态
✅ (🚀 New)	torchtitan-npu 正式开源	已支持
✅ (🚀 New)	Inductor+Autofuse 自动融合能力	已支持
✅ (🚀 New)	DeepSeek-V4-Flash模型支持	已支持
➡️ SOON	DeepSeek-V4-Pro模型支持	未支持
➡️ SOON	Muon优化器训练支持	未支持
➡️ SOON	支持FP8和A8W4的低精度训练(Ascend 950)	未支持
➡️ SOON	接入 veRL 社区，使能 RL 训练阶段 torchtitan 后端	未支持
➡️ SOON	整网性能开箱 1.0x Megatron	未支持

26年Q3概览

状态	特性	支持状态
➡️ SOON	DeepSeek-V4 训练支持 Mega MoE	未支持



欢迎体验和贡献

torchtitan-npu

基于 torchtitan 的昇腾全流程大模型训练适配插件

docs latest license BSD 3-Clause CONTRIBUTING SIG framework-adapter pypi 0.2.2.post1

简介

torchtitan-npu 定位为 torchtitan 的昇腾（Ascend）后端扩展插件，通过即插即用的硬件亲和性优化，充分释放NPU算力，助力 PyTorch native 训练在昇腾平台无缝、高效、稳定地运行。

本插件基于社区 ModelConverter 拓展机制构建，已支持多维度训练优化，涵盖 NPU融合算子、图优化、图下沉、算子自动融合、显存管理、分布式并行以及调试维测能力等等。

社群

SIG framework-adapter

SIG 例会: [sig-framework-adapter](#)



torchtitan-npu社区微信交流群



deepseek v4 续训练昇腾适配技术报告



torchtitan-npu开源仓地址



Graph-AutoFusion开源仓地址

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN